

# BDM Tabanlı Etmen Sistemleri Ontolojisi

Sami Tugal

91240000386@ogrenci.ege.edu.tr

Ege Üniversitesi Bilgisayar Mühendisliği Bölümü

Orcid: 0009-0008-6595-7053

2 Haziran 2026

## Özet

Son yıllarda büyük dil modeli (BDM) tabanlı etmenlerin yaygınlaşmasıyla birlikte LangChain, AutoGen ve CrewAI gibi çerçeveler benzer kavramları farklı isim ve soyutlamalarla sunmakta; alanda ortak, çerçeveden bağımsız ve biçimsel olarak doğrulanabilir bir kavramsal model bulunmamaktadır. Bu çalışma, BDM tabanlı etmen sistemleri için OWL 2 tabanlı bir ontoloji önermektedir. Ontoloji; etmen çekirdeği, araç kullanımı, hafıza, oturum ve mesaj akışı, model yapılandırması ile çok-etmenli orkestrasyon olmak üzere altı katmanda 47 sınıf, 51 nesne özelliği ve 79 veri özelliği içermekte ve OASIS 2 temel ontolojisini terminolojik referans olarak yeniden kullanmaktadır. Tasarım üç bağımsız yöntemle doğrulanmıştır: (i) HermiT reasoner ile tutarlılık denetimi ve tanımlı sınıflar üzerinde otomatik sınıflandırma (örn. `CodeExecutorAgent` bireyinin `AutonomousAgent` ve `MemoryAugmentedAgent` olarak çıkarsanması), (ii) Giriş'te tanımlanan altı yetkinlik sorusunun SPARQL 1.1 sorgularıyla yanıtlanması ve (iii) OOPS! aracıyla tasarım kalitesi taraması (*Critical* tuzak: 0). Ayrıca ontolojinin çerçeveden bağımsızlık iddiası, LangChain ve CrewAI çerçevelerinin gerçek sınıf hiyerarşilerinin canlı *reflection* ile çıkarılıp 37 ontoloji terimine eşlenmesiyle ampirik olarak sınanmıştır. Tüm artefaktlar (OWL dosyaları, SPARQL sorguları ve üretici betikler) açık biçimde paylaşılmaktadır.

## 1 Giriş

Büyük dil modelleri (BDM), yapay zeka alanında son yıllarda hızla gelişen ve yaygınlaşan bir teknolojidir. Doğal dil işleme, görüntü işleme ve ses işleme gibi pek çok alanda önemli başarılar elde eden bu modeller, uygun bileşenlerle birleştirildiğinde etmen tabanlı sistemlerin temelini oluşturmaktadır [11]. Bu çalışmada geliştirilen ontoloji; söz konusu bileşenleri, aralarındaki ilişkileri, karakteristik özelliklerini ve bir araya gelerek oluşturdukları etmen yapısını sistematik biçimde modellemektedir.

Ontoloji, BDM tabanlı etmen sistemlerinin kavramsal yapısını herhangi bir çerçeveden bağımsız olarak ele almakta ve aşağıdaki temel bileşenleri kapsamaktadır:

- Etmen çekirdeği ve rolü
- Araç (tool) ilişkileri

- Hafıza mekanizmaları
- Oturum ve mesaj akışı
- Çok-etmenli koordinasyon yapıları

Öte yandan aşağıdaki konular kapsam dışında bırakılmıştır:

- Transformer tabanlı modellerin eğitim süreci
- Donanım altyapısı
- Kullanıcı arayüzü tasarımı
- Çerçeveye özgü implementasyon detayları

Mevcut literatürde BDM tabanlı etmen sistemleri için genel kabul görmüş, çerçeve bağımsız bir meta-model bulunmamaktadır. LangChain, AutoGen [10] ve CrewAI gibi farklı çerçeveler, benzer kavramları farklı isim ve soyutlamalarla sunmaktadır. Bu ontoloji, söz konusu boşluğu kapatarak etmen tasarımına ilişkin ortak bir kavramsal zemin oluşturmayı amaçlamaktadır.

Ontolojinin kapsamını ve yeterliliğini doğrulamak üzere aşağıdaki yetkinlik soruları tanımlanmıştır:

- Bir etmen hangi araçlara erişebilir ve bu araçların parametrik kısıtları nelerdir?
- Bir etmen orchestrator, worker ya da her iki rolde birden mi çalışmaktadır?
- Hangi hafıza mekanizmaları hangi etmen tipleriyle ilişkilendirilebilir?
- İki etmen arasında görev devri (delegation) hangi koşullarda gerçekleşebilir?
- Bir oturum içinde üretilen mesajlar hangi etmene, role ve zaman noktasına aittir?
- Çok-etmenli bir grupta koordinasyon nasıl sağlanır?

Ontolojinin birincil hedef kitlesi şu gruplardan oluşmaktadır:

- **Yazılım mühendisleri:** Etmen tabanlı sistemleri tasarlayan ve geliştiren kişiler.
- **Alan uzmanları:** BDM tabanlı etmen sistemlerini araştıran akademisyenler ve pratisyenler.
- **Bilgi mühendisleri:** Ontoloji tabanlı akıl yürütme sistemleri geliştiren uzmanlar.

## 2 Mevcut Ontolojilerin İncelenmesi ve Temel Terimlerin Belirlenmesi

### 2.1 Literatür Taraması

Ontoloji geliştirme sürecinin bu aşamasında, öncelikle etmen sistemleri alanındaki mevcut ontolojiler incelenerek yeniden kullanım potansiyeli değerlendirilmiş, ardından ontolojide yer alacak temel terimler belirlenmiştir. Anlamsal Web vizyonu çerçevesinde, yazılım etmenlerinin insan adına bilgi sorgulayıp işleyebilmesi için makine tarafından okunabilir ve anlamsal açıdan zengin veri temsillerine ihtiyaç duyulmaktadır. Bu bağlamda W3C tarafından standartlaştırılan Web Ontology Language (OWL 2) [6], Anlamsal Web ontolojilerinin temsili için temel dil olarak kabul görmektedir.

Yapılan literatür taramasında, mevcut ontolojilerin büyük çoğunluğunun klasik çok-etmenli sistem (MAS) mimarilerini modellediği ya da IoT, blockchain gibi belirli bir uygulama alanına özgü kısıtlamalar içerdiği görülmüştür. BDM tabanlı etmen sistemleri alanında 2024–2026 döneminde çeşitli kavramsal modeller ve birleşik taksonomiler (perception/brain/planning/action/tool-use/collaboration eksenleri) önerilmiş; iş süreçleri ve kurumsal iş akışı odaklı endüstriyel çerçeveler de (örn. Skan AI'nin *Agentic Ontology of Work*'ü [4]) ortaya çıkmıştır. Akademik tarafta, ESWC 2026'da sunulan *AgentO* [3], agentic AI sistemlerini etmen (Agent), görev (Task), hedef (Goal), iş akışı deseni (WorkflowPattern/WorkflowStep), takım (Team) ve kaynak bağımlılıkları üzerinden modelleyen 15 sınıflık bir OWL/RDF ontolojisidir; LLM-güdümlü bir çıkarım süreciyle dört çerçeveden (AutoGen, CrewAI, LangGraph, Mastra AI) 66 agentic iş akışını bilgi grafiğine *örnek* (instance) düzeyinde aktararak değerlendirilmiştir. Bu çalışma ile AgentO birbirini tamamlayıcıdır ancak yöntem ve odak bakımından ayrışır: (i) **Yöntem** — AgentO iş akışlarını LLM ile örnek düzeyinde aktarırken, bu çalışma çerçevelerin sınıf hiyerarşilerini deterministik *reflection* (`__mro__`) ile *şema* düzeyinde eşleyerek terim karşılıklarını kanıtlar; (ii) **Odak** — AgentO iş akışı ve agentic-pattern düzeyinde kalırken, bu çalışma hafızayı dört alt türe (*Short/Long/Episodic/Semantic*) ve stratejilere ayırarak, ayrıca oturum-mesaj akışını, araç çağrı-sonuç zincirini, model yapılandırmasını ve asimetrik/yansısız *Delegation* ilişkisini birinci-sınıf tutarak etmenin iç mimarisine iner; (iii) **Temel ontoloji ve yeniden kullanım** — bu çalışma OASIS 2'yi `owl:imports` ile içe aktarıp `Agent`  $\sqsubseteq$  `oasis2:Agent`, `Capability`  $\sqsubseteq$  `oasis2:Behaviour` ve `Task`  $\sqsubseteq$  `oasis2:TaskDescription` hizalamalarıyla genişletir ve tanımlı sınıflar üzerinde reasoner sınıflandırması yürütür. Bu davranış, Noy ve McGuinness'in *Ontology Development 101* metodolojisinde mevcut ontolojileri “geliştir ve genişlet” (*refine and*

*extend*) etmeyi öğütleyen ve birlikte-çalışabilirlik gerektiğinde bunu bir gereklilik olarak niteleyen Adım 2 ilkesiyle birebir örtüşür [1]; AgentO ise 15 sınıfının tümünü herhangi bir temel/dış ontolojiyi miras almadan sıfırdan tanımlayarak söz konusu yeniden-kullanım adımını uygulamamaktadır. AgentO buna karşın daha geniş çerçeve kapsamına (dört çerçeve) ve kalıcı IRI altında yayımlanmış bir kaynak altyapısına sahiptir. Bununla birlikte mevcut çalışmaların çoğu ya informal taksonomi ya da iş-akışı/agentic-pattern düzeyinde kalmakta; hafıza katmanlaması, oturum-mesaj akışı ve araç çağrı zincirini birinci-sınıf ve reasoner-doğrulanabilir biçimde, OASIS 2 gibi bir temel ontolojiyle hizalı olarak bir araya getiren çerçeveden bağımsız bir BDM etmen ontolojisi henüz yaygın değildir. Bu çalışma söz konusu keşimi hedeflemekte ve ayrıca canlı *reflection* ile ampirik kapsama doğrulaması sunarak farklılaşmaktadır. BDM etmenlerini karakterize eden *reasoning-acting* döngüsü [8], dış araç çağırma yeteneği [9] ve hafıza katmanı [12] gibi temel mekanizmalar, mevcut formal etmen ontolojilerinde doğrudan kavramsal karşılığa sahip değildir.

**FIPA Standartları:** Foundation for Intelligent Physical Agents [5] tarafından geliştirilen Agent Communication Language (ACL), çok-etmenli sistemlerde iletişim standardı olarak öne çıkmaktadır. FIPA-ACL, mesaj yapısını *performative* (iletişimsel eylem tipi), *sender*, *receiver* ve *content* parametreleri üzerinden tanımlar; bu standartlarla beraber tanımlanan ACL Performatif, Directory Facilitator kavramları günümüzdeki BDM tabanlı sistemler için geçerli olmakla birlikte, BDM tabanlı sistemlerin gerektirdiği *Memory*, *Tool* ve *Session* kavramlarını içermemektedir.

İncelenen çalışmalar arasında en kapsamlı aday olan OASIS 2 [2], OWL 2 tabanlı bir temel ontoloji olup davranışçı (behaviouristic) bir yaklaşım benimsemektedir. Etmenleri dört temel sınıf üzerinden modellemektedir:

- **Agent:** Tüm etmen bireylerini kapsayan temel sınıf; *hasBehaviour* özelliği ile davranışlara bağlanır.
- **Behaviour:** Etmenin ulaşabileceği hedefleri toplayan yapı; *consistsOfGoalDescription* özelliği ile hedeflere bağlanır.
- **GoalDescription:** Etmenin gerçekleştirebileceği görevleri içeren hedef tanımları; *dependsOn* özelliği ile bağımlılık ilişkilerini ifade eder.
- **TaskDescription:** Etmenlerin gerçekleştirdiği atomik operasyonları tanımlayan sınıf.

OASIS 2, etmen davranışlarının anlamsal temsilini sağlamakta ve etmenlerin tak-cıkart biçiminde işbirlikçi ortamlara katılmasını mümkün kılmaktadır. Ancak blockchain ve IoT bağlamına özgü kısıtlamalar içermesi ve BDM tabanlı sistemler için kritik olan *Session*, *Memory* ve *Tool* katmanlarını kapsamaması nedeniyle doğrudan içe aktarılması uygun görülmemiştir. Bu değerlendirmeler doğrultusunda ontoloji sıfırdan geliştirilecek; OASIS 2 bünyesindeki

**Agent** ve **Task** sınıflarının kavramsal tanımları ile FIPA standartlarındaki iletişim protokolleri yalnızca terminolojik referans olarak ele alınacaktır.

İncelenen yaklaşımların BDM tabanlı etmen sistemleri açısından kapsama düzeyi Tablo 1'de özetlenmiştir. Tablo, hiçbir mevcut yaklaşımın hafıza, araç kullanımı, oturum yönetimi ve orkestrasyon katmanlarını aynı anda OWL 2 tabanlı ve reasoner-doğrulanabilir biçimde, çerçeveden bağımsız olarak kapsamadığını; bu çalışmanın hedeflediği boşluğun bu kesişimde yer aldığını göstermektedir.

## 2.2 Önemli Terimlerin Belirlenmesi

Mevcut ontolojilerin yetersiz kaldığı noktalar belirlendikten sonra, ontolojide ifade edilmesi hedeflenen kavramlar sınıf ve slot ayrımı gözetilmeksizin listelenmiştir [1]. Toplam 37 terim altı grupta sunulmaktadır:

### Çekirdek Varlıklar:

- **Agent:** Karar veren, yönlendiren ve diğer bileşenleri orkestre eden temel varlık.
- **AgentGroup:** Ortak bir amaç doğrultusunda birlikte çalışan etmenlerin mantıksal organizasyonu.
- **AgentTemplate:** Somut bir etmen örneği oluşturmak için kullanılan soyut yapılandırma şablonu.
- **Role:** Bir etmenin sistem içindeki işlevsel konumunu ve sorumluluğunu belirleyen tanım.
- **Policy:** Etmenin karar ve davranışlarını kısıtlayan veya yönlendiren kurallar bütünü.

### Etkileşim ve Akış:

- **Session:** Bir kullanıcı ile etmen arasındaki belirli bir etkileşim sürecini kapsayan bağlam birimi.
- **Message:** Kullanıcıdan ya da etmenden gelen, role ve sıraya sahip tekil iletişim birimi.
- **Observation:** Bir oturum sırasında üretilen token kullanımı, gecikme ve hata gibi sistem düzeyindeki ölçüm kayıtları.
- **Context:** Etmenin karar üretirken değerlendirdiği anlık bağlam penceresi.

### Yetenek Katmanı:

- **Tool:** Etmen tarafından çağrılabilen, tanımlı bir işi yerine getiren dışsal fonksiyon veya sistem bileşeni.
- **ToolCall:** Etmenin bir aracı belirli parametrelerle çağırma eylemi.
- **ToolResult:** Bir araç çağırısının ürettiği çıktı ya da hata durumu.
- **Resource:** Etmenin erişebildiği dosya, veritabanı veya harici servis gibi veri kaynakları.
- **Action:** Etmenin bir karar sonucunda gerçekleştirdiği somut işlem.
- **Capability:** Bir etmenin sahip olduğu yetenek kümesinin soyut tanımı.

### Hafıza Katmanı:

- **Memory:** Etmenin geçmiş etkileşimlere ve bilgiye erişimini sağlayan genel hafıza altyapısı.

- **ShortTermMemory:** Aktif oturum süresince geçerli olan kısa vadeli bağlam hafızası.
- **LongTermMemory:** Oturumlar arasında kalıcı olarak saklanan uzun vadeli bilgi deposu.
- **EpisodicMemory:** Geçmiş etkileşimlere ait olaysal kayıtları saklayan hafıza türü.
- **SemanticMemory:** Alan bilgisi ve olgusal bilgiyi temsil eden yapılandırılmış bilgi tabanı.
- **MemoryStrategy:** Hangi bilginin ne zaman saklanacağını ve geri çağrılacağını belirleyen strateji.

### Model ve Yapılandırma:

- **LLMModel:** Etmenin çıktı üretmek için kullandığı büyük dil modeli.
- **Prompt:** Modele gönderilen ve davranışı yönlendiren girdi metni.
- **SystemPrompt:** Etmenin genel davranışını, rolünü ve kısıtlamalarını tanımlayan sabit yönerge.
- **UserPrompt:** Kullanıcı ya da çağırıcı sistem tarafından sağlanan, etmene yönlendirilen anlık talep veya soru.
- **PromptTemplate:** Dinamik değişkenler içeren ve çalışma zamanında doldurulabilen prompt şablonu.
- **Parameter:** Modelin sıcaklık ve maksimum token gibi çalışma zamanı ayarlarını tanımlayan değer.
- **ModelConfiguration:** Bir etmen için kullanılacak modeli ve parametrelerini bir arada tanımlayan yapılandırma nesnesi.

### Orkestrasyon ve İletişim:

- **Task:** Etmenin tamamlaması beklenen, tanımlı giriş ve çıkışa sahip iş birimi.
- **Subtask:** Bir görevin daha küçük ve bağımsız olarak yürütülebilir alt parçası.
- **Plan:** Bir görevi tamamlamak için belirlenen sıralı adımlar dizisi.
- **Workflow:** Birden fazla etmen veya aracın koordineli biçimde yürüttüğü süreç akışı.
- **Delegation:** Bir etmenin görevi başka bir etmene devretme ilişkisi.
- **ExecutionContext:** Bir görevin çalıştırıldığı ortam bilgisini ve durumunu kapsayan bağlam nesnesi.
- **Channel:** Etmenler veya etmen ile kullanıcı arasındaki iletişim kanalının soyut tanımı.
- **Protocol:** Etmenler arası mesaj alışverişini düzenleyen iletişim kuralları bütünü.
- **MessageRole:** Bir mesajın sistemdeki rolünü belirleyen etiket (**user**, **assistant**, **system** vb.).

Toplam 37 terim belirlenmiştir. Bu terimlerin bir kısmı ilerleyen aşamalarda bağımsız sınıflar olarak tanımlanacak, bir kısmı ise sınıflara ait slot ya da facet olarak modelleneyecektir.

## 3 Ontoloji Tasarımı

Bu bölümde ontolojinin sınıf hiyerarşisi, nesne ve veri özellikleri, kısıtlar ve bireyler ele alınmaktadır. OWL 2 implementasyonu RDF/XML formatında sunulmaktadır.

| Ontoloji / Yaklaşım     | Hafıza | Araç | Oturum | Orkestrasyon / Delegasyon | OWL 2 + Reasoner | Çerçeve-bağımsız |
|-------------------------|--------|------|--------|---------------------------|------------------|------------------|
| FIPA-ACL [5]            | –      | –    | –      | K                         | –                | ✓                |
| OASIS 2 [2]             | –      | –    | –      | K                         | ✓                | K                |
| Klasik MAS ontolojileri | –      | –    | –      | ✓                         | K                | ✓                |
| BDM etmen taksonomileri | ✓      | ✓    | K      | ✓                         | –                | ✓                |
| AgentO [3]              | K      | ✓    | –      | ✓                         | ✓                | ✓                |
| <b>Bu çalışma</b>       | ✓      | ✓    | ✓      | ✓                         | ✓                | ✓                |

Tablo 1: Mevcut yaklaşımların BDM-etmen kavramları açısından karşılaştırılması (✓: tam kapsam, K: kısmi / dolaylı, –: yok). AgentO tek bir **Memory** sınıfı içerir (hafıza alt türleri yoktur) ve iş akışı/agent-pattern modellemesine odaklanır; satır, yayımlanmış tanım ve kamuya açık ontoloji deposuna göre değerlendirilmiştir.

### 3.1 Sınıf Hiyerarşisi

Ontoloji 37 terimi altı katmanda organize etmektedir. Her katman işlevsel bir sorumluluğa karşılık gelir.

#### 1. Çekirdek Varlıklar

- `Agent`  $\sqsubseteq$  `oasis:Agent`
- `AgentGroup`, `AgentTemplate`
- `Role`, `Policy`
- `ModelConfiguration`

#### 2. Etkileşim ve Akış

- `Session`, `Context`, `Observation`
- `Message`  $\sqsubseteq$  `AgentMessage`, `UserMessage`
- `MessageRole` (oneOf: `User`, `Assistant`, `System`, `ToolRole`)

#### 3. Yetenek Katmanı

- `Capability`  $\sqsubseteq$  `oasis:Behaviour`
- `Tool`, `ToolCall`, `ToolResult`
- `Resource`, `Action`

#### 4. Hafıza Katmanı

- `Memory`  $\sqsubseteq$  `ShortTermMemory`, `LongTermMemory`, `EpisodicMemory`, `SemanticMemory`
- `MemoryStrategy`  $\sqsubseteq$  `CompressionStrategy`, `EvictionStrategy`, `RetrievalStrategy`

#### 5. Model ve Yapılandırma

- `LLMModel`, `Parameter`
- `Prompt`  $\sqsubseteq$  `SystemPrompt`, `UserPrompt`
- `PromptTemplate`

#### 6. Orkestrasyon ve İletişim

- `Task`  $\sqsubseteq$  `Subtask`; `Plan`, `Workflow`
- `Delegation`, `ExecutionContext`
- `Channel`, `Protocol` (örn. FIPA-ACL, JSON-RPC, MCP [13])

Şekil 1, ontolojinin merkezindeki `Agent` sınıfının diğer temel sınıflara nesne özellikleri aracılığıyla nasıl bağlandığını özetlemektedir.

### 3.2 Tanımlı Sınıflar (Faz 4)

OWL 2 eşdeğerlik aksiyomuyla üç özelleşmiş etmen tipi türetilmiştir. Bu tanımlar bir reasoner aracılığıyla bireylerin otomatik sınıflandırılmasını sağlar:

$$\begin{aligned}
\text{AutonomousAgent} &\equiv A \sqcap \exists hM.M \sqcap \exists mTC.TC \\
\text{MemoryAugmentedAgent} &\equiv A \sqcap \exists hM.LTM \\
\text{PlanningAgent} &\equiv A \sqcap \exists cP.P \sqcap \neg(\exists mTC.TC)
\end{aligned}$$

Kısaltmalar:  $A$ =Agent,  $hM$ =hasMemory,  $M$ =Memory,  $mTC$ =makesToolCall,  $TC$ =ToolCall,  $LTM$ =LongTermMemory,  $cP$ =createsPlan,  $P$ =Plan.

Ek olarak iki birleşim (union) sınıfı tanımlanmıştır:

- `ExecutableEntity`  $\equiv$  `Task`  $\sqcup$  `Workflow`
- `MemoryContent`  $\equiv$  `Message`  $\sqcup$  `Session`

`MemoryContent`, `hasMemory` ve hafıza saklama özelliklerinin (`storesMessage`, `recordsSession`) ortak range'i olarak işlev görür: hafıza katmanı için *neyin hatırlanabilir bir içerik olduğunu* tek bir tip altında yakalar. Böylece `Session` bireyleri (oturum-düzeyi epizotlar) ile `Message` bireyleri (mesaj-düzeyi parçacıklar) farklı granüleritede oldukları halde hafıza sorgularında birlikte ele alınabilir.

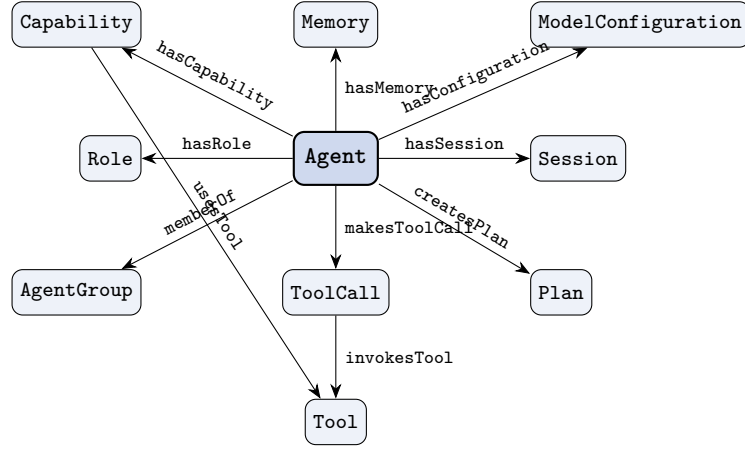
Ve kapalı liste (oneOf) ile tanımlı bir çalıştırma modu:

- `ExecutionMode`  $\equiv$  {`Sync`, `Async`}

**Toplam sınıf dökümü.** Ontoloji 47 OWL sınıfı içerir: Bölüm 2.2'deki 37 temel terimin OWL karşılıklarına ek olarak Faz 4'te tanımlanan üç eşdeğerlik sınıfı (`AutonomousAgent`, `MemoryAugmentedAgent`, `PlanningAgent`), iki birleşim sınıfı (`ExecutableEntity`, `MemoryContent`), bir oneOf sınıfı (`ExecutionMode`), `Message` altında iki alt tip (`AgentMessage`, `UserMessage`) ve `MemoryStrategy` altında üç alt tip (`Compression-`, `Eviction-`, `RetrievalStrategy`) yer alır.

### 3.3 Nesne Özellikleri

Tablo 2, ontolojideki tüm nesne özelliklerini domain, range ve aksiyom bilgisiyle özetlemektedir.



Şekil 1: Agent sınıfını merkez alan temel nesne özelliği ilişkileri. *hasConfiguration exactly 1*, *hasCapability* ise *someValuesFrom* kısıtıyla daraltılmıştır.

| Özellik           | Domain        | Range         | Aksiyom               |
|-------------------|---------------|---------------|-----------------------|
| accessesResource  | Tool          | Resource      | -                     |
| capabilityOf      | Capability    | Agent         | inverseOf             |
| coordinatesVia    | AgentGroup    | Workflow      | -                     |
| createsPlan       | Agent         | Plan          | -                     |
| delegatedBy       | Agent         | Delegation    | inverseOf             |
| delegatedTask     | Delegation    | Task          | Functional            |
| delegatesTo       | Delegation    | Agent         | Asymm., Irref.        |
| delegator         | Delegation    | Agent         | Functional            |
| followsProtocol   | Ch./Del./Wf.  | Protocol      | -                     |
| forRole           | Prompt        | MessageRole   | -                     |
| generatedBy       | AgentMsg.     | Agent         | Functional            |
| hasAction         | ToolCall      | Action        | -                     |
| hasCapability     | Agent         | Capability    | someValuesFrom        |
| hasConfiguration  | Agent         | ModelConf.    | exactly 1             |
| hasContext        | Agent         | Context       | -                     |
| hasMember         | AgentGroup    | Agent         | inverseOf             |
| hasMemory         | Agent         | Memory        | -                     |
| hasMemoryStrategy | Memory        | MemStrategy   | -                     |
| hasMessage        | Session       | Message       | inverseOf             |
| hasMessageRole    | Message       | MessageRole   | Functional            |
| hasObservation    | Session       | Observation   | inverseOf             |
| hasParameter      | ModelConf.    | Parameter     | -                     |
| hasPlan           | Workflow      | Plan          | -                     |
| hasPolicy         | Agent         | Policy        | -                     |
| hasPrompt         | ModelConf.    | Prompt        | -                     |
| hasRole           | Agent         | Role          | -                     |
| hasSession        | Agent/Group   | Session       | -                     |
| hasSharedMemory   | AgentGroup    | Memory        | -                     |
| hasStep           | Plan          | Task          | allValuesFrom         |
| instantiates      | Agent         | AgentTemplate | -                     |
| invokedBy         | ToolCall      | Agent         | -                     |
| invokesTool       | ToolCall      | Tool          | someValuesFrom        |
| madeBy            | Observation   | Agent         | -                     |
| makesToolCall     | Agent         | ToolCall      | -                     |
| memberOf          | Agent         | AgentGroup    | -                     |
| observedIn        | Observation   | Session       | inverseOf             |
| partOfSession     | Message       | Session       | inverseOf             |
| playedBy          | Role          | Agent         | -                     |
| producesResult    | ToolCall      | ToolResult    | -                     |
| promptOf          | Prompt        | ModelConf.    | -                     |
| recordsSession    | EpisodicMem.  | Session       | -                     |
| resultOf          | ToolResult    | ToolCall      | Functional            |
| runsIn            | Task          | ExecutionCtx. | Functional            |
| sessionOf         | Session       | Ag./AgGr. □   | -                     |
| storesMessage     | ShortTermMem. | Message       | -                     |
| subtaskOf         | Subtask       | Task          | Func., Asymm., Irref. |
| toolFor           | Tool          | Capability    | inverseOf             |
| triggeredBy       | Delegation    | Policy        | Functional            |
| usesModel         | ModelConf.    | LLMModel      | -                     |
| usesTool          | Capability    | Tool          | -                     |
| viaChannel        | Workflow      | Channel       | -                     |

Tablo 2: Tüm nesne özellikleri (51 adet)

### 3.4 Agent Varlığının Kapsamı

Agent, ontolojinin merkez sınıfıdır. Sınıf `oasis:Agent`'tan türetilmiş ve iki zorunlu OWL kısıtıyla daraltılmıştır: her etmen en az bir yeteneğe (*someValuesFrom hasCapability*) ve tam olarak bir konfigürasyona (*exactly 1 hasConfiguration*) sahip olmalıdır.

### Nesne Özellikleri

| Özellik          | Yön                | Aksiyom / Anlam |
|------------------|--------------------|-----------------|
| hasCapability    | Agent → Capability | someValuesFrom  |
| hasConfiguration | Agent → ModelConf  | exactly 1       |
| hasContext       | Agent → Context    | -               |
| hasMemory        | Agent → Memory     | -               |
| hasPolicy        | Agent → Policy     | -               |
| hasRole          | Agent → Role       | -               |
| hasSession       | Agent → Session    | -               |
| makesToolCall    | Agent → ToolCall   | -               |
| memberOf         | Agent → AgentGroup | -               |
| createsPlan      | Agent → Plan       | -               |
| delegatedBy      | Agent → Delegation | inverseOf       |
| capabilityOf     | Capability → Agent | inverseOf       |
| generatedBy      | AgentMsg. → Agent  | Functional      |
| delegatesTo      | Delegation → Agent | Asymm., Irref.  |
| delegator        | Delegation → Agent | Functional      |
| hasMember        | AgentGroup → Agent | inverseOf       |

Tablo 3: Agent nesne özellikleri

### Veri Özellikleri

| Özellik      | Tip      | Açıklama                      |
|--------------|----------|-------------------------------|
| agentId      | string   | Benzersiz kimlik (Functional) |
| agentName    | string   | İnsan okunabilir ad           |
| agentVersion | string   | Sürüm etiketi                 |
| isActive     | boolean  | Çalışma durumu                |
| createdAt    | dateTime | Oluşturulma zamanı            |

Tablo 4: Agent veri özellikleri

### Ayrıklık Kısıtları

Agent, Memory, Message, Session ve Tool sınıfları `owl:AllDisjointClasses` aksiyomuyla birbirinden tamamen ayrılmıştır. Bu kısıt, bir bireyin aynı anda birden fazla temel sınıfa üye olmasını önler.

### Somut Bireyler

Çok-etmenli araştırma sistemi senaryosunda dört bireysel etmen örneklenmiştir:

- **OrchestratorAgent**: GPT-4o kullanan koordinatör; araştırma oturumunu yönetir.
- **ResearchPlannerAgent**: Plan üreten, araç çağrısı yapmayan; reasoner tarafından **PlanningAgent** olarak çıkarılabilir.
- **CodeExecutorAgent**: Uzun vadeli ve epizodik belleğe sahip, araç çağrısı yapan; **AutonomousAgent** olarak çıkarılabilir.
- **ReviewerAgent**: Llama-3 kullanan, dosya okuma yeteneğine sahip değerlendirici etmen.



## 4 Doğrulama ve Test

Ontolojinin tasarım aşamasında tanımlanan kavramsal yapının iç tutarlılığı ve Bölüm 1’de belirlenen yetkinlik sorularını yanıtlama kapasitesi, üç ayrı doğrulama yöntemiyle sınanmıştır. İlk yöntemde HermiT reasoner [14] aracılığıyla aksiyom kümesinin tutarlılığı kontrol edilmiş ve tanımlı sınıflar üzerinde otomatik sınıflandırma yürütülmüştür. İkinci yöntemde yetkinlik sorularının her biri SPARQL 1.1 [7] ile sorgulanmış ve elde edilen sonuçlar beklenen sonuçlarla karşılaştırılmıştır. Üçüncü yöntemde ise ontolojinin modelleme kalitesi OOPS! (Ontology Pitfall Scanner!) [15] aracıyla taranmıştır.

### 4.1 Reasoner Tutarlılık Kontrolü

HermiT reasoner, geliştirilen ontolojiyi (76 birey, 47 sınıf, 51 nesne özelliği) OASIS 2 ile birlikte yüklemiş ve tutarlılık analizine tabi tutmuştur. Sonuçlar üç açıdan değerlendirilmektedir:

- Ontoloji **tutarlıdır** (*consistent*); hiçbir sınıf owl:Nothing ile eşdeğer olarak çıkarsanmamıştır.
- AllDisjointClasses aksiyomları (Agent, Memory, Message, Session, Tool) sağlanmış; herhangi bir birey iki ayrıık sınıfa eşzamanlı üyelik göstermemektedir.
- Tanımlı sınıflar üzerinde reasoner, CodeExecutorAgent bireyini hem AutonomousAgent hem MemoryAugmentedAgent olarak otomatik sınıflandırmıştır.

Sınıflandırma sonuçları, ontolojinin Açık Dünya Varsayımına (Open World Assumption, OWA) tabi olduğunu da somut biçimde ortaya koymuştur. Örneğin PlanningAgent tanımı  $Agent \sqcap \exists createsPlan.Plan \sqcap \neg(\exists makesToolCall.ToolCall)$  biçiminde negasyon içermektedir; reasoner, ResearchPlannerAgent bireyinin araç çağırısı yapmadığını ABox kapatılmadığı sürece çıkarsayamamaktadır. Bu davranış bir hata değil, OWA’nın beklenen sonucudur ve ontolojinin doğru biçimde modellendiğine işaret etmektedir.

Bu kapamanın etkisini somut olarak göstermek için ek bir kontrollü deney yürütülmüştür. ResearchPlannerAgent bireyine, araç çağırısı yapmadığını bildiren bir yerel kapama aksiyomu — OWL Functional sözdiziminde  $ClassAssertion(ObjectComplementOf(ObjectSomeValuesFrom(:makesToolCall :ToolCall)) :ResearchPlannerAgent)$  — eklenmiş ve ontoloji HermiT ile yeniden çıkarıma sokulmuştur. Kapama öncesinde PlanningAgent tanımlı sınıfının hiçbir üyesi çıkarsanamazken (boş sonuç kümesi), kapama sonrasında reasoner ResearchPlannerAgent bireyini beklediği gibi PlanningAgent olarak sınıflandırmıştır. Bu sonuç hem PlanningAgent tanımının doğruluğunu hem de OWA kaynaklı çıkarsanamama davranışının yerel kapama (*local closed-world*) ile aşılabildiğini doğrulamakta; böylece Faz 4’te tanımlanan

üç özelleşmiş etmen tipinin üçü de (AutonomousAgent, MemoryAugmentedAgent, PlanningAgent) çalışır biçimde gösterilmiş olmaktadır.

### 4.2 SPARQL ile Yetkinlik Sorusu Doğrulaması

Bölüm 1’de tanımlanan altı yetkinlik sorusunun her birine karşılık tek bir SPARQL sorgusu hazırlanmış ve reasoner çıkarımlarıyla zenginleştirilmiş ontoloji üzerinde çalıştırılmıştır. Sorguların tamamı ve ham CSV çıktıları Ek A’da sunulmaktadır; bu bölümde üç temsili sorgu ön plana çıkarılmaktadır.

#### Sorgu Q1: Araç Erişimi ve Kaynak Bağı

Birinci yetkinlik sorusu, etmenlerin hangi araçları kullanabildiğini ve bu araçların eriştiği dış kaynakları sorgulamaktadır:

```
1 PREFIX : <http://www.ege.edu.tr/cs/ontology/llm-agent#>
2 SELECT ?agent ?tool ?resource WHERE {
3   ?agent a :Agent ;
4         :makesToolCall ?call .
5   ?call :invokesTool ?tool .
6   OPTIONAL { ?tool :accessesResource ?
7               resource . }
8 }
```

Sorgu, yalnızca araç çağırma yeteneği aktif olarak modellenen CodeExecutorAgent bireyini sonuç kümesine almıştır. CodeInterpreterTool bireyi CodeResource, DBResource ve FileResource kaynaklarına; WebSearchTool bireyi ise WebResource kaynağına erişmektedir.

#### Sorgu Q3: Hafıza ve Etmen Tipi İlişkisi

Üçüncü yetkinlik sorusu, hafıza mekanizmalarının hangi etmenlerle ilişkilendirildiğini sorgular. Aşağıdaki sorgu, somut etmen bireyleri ile bağlı oldukları dört hafıza türünü listelemektedir:

```
1 PREFIX : <http://www.ege.edu.tr/cs/ontology/llm-agent#>
2 SELECT ?agent ?memoryType WHERE {
3   ?agent a :Agent ;
4         :hasMemory ?mem .
5   ?mem a ?memoryType .
6   FILTER(?memoryType IN (:
7       ShortTermMemory,
7       LongTermMemory, :EpisodicMemory,
8       SemanticMemory))
8 }
```

Sorgunun ürettiği sonuç şu şekildedir:

| Etmen                | Hafıza Türü     |
|----------------------|-----------------|
| CodeExecutorAgent    | EpisodicMemory  |
| CodeExecutorAgent    | LongTermMemory  |
| CodeExecutorAgent    | SemanticMemory  |
| ResearchPlannerAgent | ShortTermMemory |

Tablo 5: Q3 sorgusu sonucu

Elde edilen sonuçlar, `CodeExecutorAgent` bireyinin uzun vadeli, epizodik ve semantik hafızaya sahip olması nedeniyle `MemoryAugmentedAgent` tanımlı sınıfının koşulunu sağladığını teyit etmektedir.

#### Sorgu Q4: Görev Devri Koşulları

Dördüncü yetkinlik sorusu, etmenler arasındaki delegasyon yapısını sorgulamaktadır:

```
1 PREFIX : <http://www.ege.edu.tr/cs/ontology/llm-agent#>
2 SELECT ?delegator ?delegatee ?task
3 WHERE {
4   ?d a :Delegation ;
5   :delegator ?delegator ;
6   :delegatesTo ?delegatee ;
7   :delegatedTask ?task .
8 }
```

Sorgu sonucunda iki delegasyon ilişkisi tespit edilmiştir: `OrchestratorAgent`, `SummaryTask` görevini `ReviewerAgent`'a devretmiş; `ResearchPlannerAgent` ise `CodeAnalysisTask` görevini `CodeExecutorAgent`'a aktarmıştır. `delegatesTo` özelliğinin asimetrik ve yansız (*irreflexive*) olarak tanımlanması nedeniyle, hiçbir etmenin kendisine ya da kendisine devreden bir etmene görev devretmesi mümkün değildir.

#### 4.3 OOPS! ile Tasarım Kalitesi Taraması

Biçimsel tutarlılığın ötesinde, ontolojinin modelleme kalitesi OOPS! (Ontology Pitfall Scanner!) [15] çevrimiçi aracıyla değerlendirilmiştir. OOPS!, ontolojileri yaygın modelleme tuzakları (*pitfalls*) açısından üç önem düzeyinde (*Critical*, *Important*, *Minor*) inceleyen, reasoner'dan bağımsız bir araçtır; bu yönüyle mantıksal tutarlılık denetimini tasarım kalitesi denetimiyle tamamlar. Tarama, kaynak ontoloji (`agentic-ontology.rdf`) üzerinde çalıştırılmıştır.

Sonuçlar Tablo 6'da özetlenmektedir. **Hiçbir *Critical* tuzak tespit edilmemiş; ontoloji taramadan başarıyla geçmiştir.** Raporlanan sınırlı sayıda tuzak ya anlam taşımayan (kozmetik) niteliktedir ya da harici import'tan kaynaklanan yan etkilerdir.

| Kod | Önem      | Pitfall                     | Eleman |
|-----|-----------|-----------------------------|--------|
| P11 | Important | Eksik domain/range          | 2      |
| P34 | Important | Tipsiz sınıf                | 3      |
| P08 | Minor     | Eksik açıklama (annotation) | 3      |
| P13 | Minor     | Bildirilmemiş ters ilişki   | 27     |

Tablo 6: OOPS! pitfall tarama sonuçları (*Critical*: 0). P08 ilk taramada 180 öğeyi etkilemekteydi; annotation katmanı eklendikten sonra yalnızca üç OASIS 2 dış öğesi kalmıştır.

Bulguların yorumu:

- **P34 (3):** `oasis:Agent`, `oasis:Behaviour` ve `oasis:TaskDescription` sınıflarının tipsiz görün-

mesi, taramanın OASIS 2 import'unu çözemesinden kaynaklanan bir yan etkidir; ontolojinin kendi kusuru değildir.

- **P11 (2):** `hasDescription` ve `updatedAt`, birden çok sınıfta kullanılabilmesi için bilinçli olarak domain'i açık bırakılmış jenerik metadata özellikleridir.
- **P08 (3):** İlk taramada 47 sınıf, 51 nesne özelliği ve 79 veri özelliği için `rdfs:label` ile `rdfs:comment` eksik bulunmuştu (toplam 177 yerel öğe; OOPS! ayrıca üç OASIS 2 sınıfını da raporlayarak sayıyı 180'e taşımıştı). İkinci taramaya kadar tüm 177 yerel öğeye Türkçe açıklama ve İngilizce etiket annotation'ları enjekte edilmiştir; bu işlem `ontology/add_annotations.py` betiği ile tekrarlanabilir. Kalan üç öğe yine OASIS 2 dış sınıflardır ve P34 ile aynı gerekçeye dahildir.
- **P13 (27):** Bildirilmemiş ters ilişki önerileridir; heuristik öneri niteliğinde, anlamsal doğruluğu etkilemeyen *Minor* düzeyinde bir tuzaktır. Ontolojide zaten altı `inverseOf` çifti açıkça tanımlıdır; geri kalan öneriler tek-yönlü modellenmesi tasarımca tercih edilen özellikler (örn. `hasConfiguration`, `hasMessageRole`) işaret etmektedir.

#### 4.4 Doğrulama Özeti

Altı yetkinlik sorusu, altı SPARQL sorgusuyla doğrulanmış; reasoner tarafından üretilen iki yeni `rdf:type` aksiyomu (`CodeExecutorAgent` için `AutonomousAgent` ve `MemoryAugmentedAgent`) `ABox`'a eklenmiştir. OOPS! pitfall taraması ise hiçbir *Critical* tuzak tespit etmeyerek ontolojinin tasarım kalitesini teyit etmiştir. Tüm sorguların ham çıktıları ve ek sınıflandırma raporu sırasıyla Ek A ve Ek C'de yer almaktadır. Bu iki yöntemin yanında, ontolojinin çerçeveden bağımsızlık iddiası üçüncü ve ampirik bir doğrulama (LangChain ve CrewAI çerçevelerinin gerçek sınıf hiyerarşilerinin ontolojiye eşlenmesiyle) Ek B'de sinanmıştır.

### 5 Sonuç ve Değerlendirme

Bu çalışmada, BDM tabanlı etmen sistemleri için OWL 2 tabanlı, reasoner-doğrulanabilir ve çerçeveden bağımsız bir biçimsel kavramsal model geliştirilmiştir. Mevcut literatürdeki ontolojiler klasik çok-etmenli sistem mimarilerine veya IoT/blockchain gibi alana özgü senaryolara odaklanmakta; aynı dönemde önerilen taksonomi ve kurumsal iş akışı çerçeveleri ise informal kalmakta ve hafıza, araç kullanımı, oturum yönetimi ile orkestrasyon katmanlarını birinci-sınıf ve reasoner-denetlenabilir biçimde kapsamamaktadır (Tablo 1). Geliştirilen ontoloji, söz konusu boşluğu doldurarak farklı çerçevelerin ortak bir anlamsal zemin üzerinde karşılaştırılabilmesini ve birlikte çalışabilirliğini sağlamayı hedeflemektedir.

Çalışmanın somut çıktıları şunlardır:

- Altı katmanda 47 sınıf, 51 nesne özelliği ve 79 veri

özellığı içeren, HermiT ile tutarlılığı doğrulanmış OWL 2 ontolojisi.

- Altı yetkinlik sorusunu yanıtlayan ve reasoner-zenginleştirilmiş ontoloji üzerinde çalıştırılan SPARQL sorgu kataloğu (Ek A).
- LangChain ve CrewAI çerçevelerinin gerçek sınıf hiyerarşilerinin canlı reflection ile ontolojiye eşlenmesi (Ek B).
- OOPS! taramasında *Critical* tuzak içermeyen, etiket ve açıklama annotation'larıyla zenginleştirilmiş tasarım.

## 5.1 Sınırlılıklar

- Ampirik doğrulama, tek bir çok-etmenli araştırma senaryosu (76 birey) üzerinde yürütülmüştür; ontolojinin ölçeklenebilirliği ve kapsayıcılığı daha geniş ve çeşitli ABox'larla sınanmalıdır.
- Tanımlı sınıflardan **PlanningAgent**, negasyon içerdiğinden ancak yerel kapama aksiyomuyla çıkarsanabilmektedir (Bölüm 4.1); varsayılan ABox kapaması henüz ontolojiye dahil değildir.
- OASIS 2 dış ontolojisine bağımlılık, tarama araçları import'u çözemediğinde yan etkilere (örn. OOPS! P34) yol açmaktadır.
- Çerçeve eşlemesi iki çerçeveye (LangChain, CrewAI) sınırlıdır; AutoGen, Semantic Kernel gibi diğer çerçeveler henüz kapsanmamıştır.

## 5.2 Gelecek Çalışma

Ontolojinin modellediği alan oldukça dinamik ve hızlı gelişen bir alandır. Yeni etmen mimarileri, araç entegrasyonları ve koordinasyon desenleri ortaya çıktıkça ontolojinin de güncellenmesi gerekmektedir. Bu sebeple ontoloji genişletilebilir bir yapıda tasarlanmış olup gelecekte şu yönlerde sürdürülmesi planlanmaktadır: (i) eşleme kapsamının AutoGen ve diğer çerçevelere genişletilmesi; (ii) daha büyük ve çeşitli senaryolarla nicel kapsama değerlendirmesi; (iii) yerel kapama ve SWRL kuralları aracılığıyla tanımlı sınıf çıkarımlarının güçlendirilmesi; (iv) ontolojinin kalıcı bir IRI altında yayımlanması ve topluluk katkısına açılması.

## Kaynakça

- [1] Noy, N. F. and McGuinness, D. L. (2001). *Ontology Development 101: A Guide to Creating Your First Ontology*. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05.
- [2] Bella, G., Cantone, D., Longo, C. F., Nicolosi-Asmundo, M. and Santamaria, D. F. (2023). *The Ontology for Agents, Systems and Integration of Services: OASIS version 2*. Intelligenza Artificiale, vol. 17, no. 1, pp. 51–62. DOI: 10.3233/IA-230002.
- [3] Ekelhart, A., Kurniawan, K., Ekaputra, F. J. and Kiesling, E. (2026). *AgentO: An Ontology for Modeling Agentic AI Systems*. In: The Semantic Web — ESWC 2026, Lecture Notes in Computer Science, Springer, Cham, pp. 298–320. DOI: 10.1007/978-3-032-25159-6\_16.
- [4] Garg, M. (2026). *Agentic Ontology of Work (AOW)*. Skan AI, White paper. <https://www.skan.ai/whitepapers/agentic-ontology-of-work>.
- [5] FIPA (2002). *FIPA ACL Message Structure Specification*. Foundation for Intelligent Physical Agents, Document No. SC00061G.
- [6] Hitzler, P., Krötzsch, M., Parsia, B., Patel-Schneider, P. F. and Rudolph, S. (eds.) (2012). *OWL 2 Web Ontology Language Document Overview (Second Edition)*. W3C Recommendation, 11 December 2012. <https://www.w3.org/TR/owl2-overview/>.
- [7] Harris, S. and Seaborne, A. (eds.) (2013). *SPARQL 1.1 Query Language*. W3C Recommendation, 21 March 2013. <https://www.w3.org/TR/sparql11-query/>.
- [8] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR). arXiv:2210.03629.
- [9] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N. and Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2302.04761.
- [10] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D. and Wang, C. (2024). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. Conference on Language Modeling (COLM 2024). arXiv:2308.08155.
- [11] Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., Fan, X., Wang, X., Xiong, L., Zhou, Y., Wang, W., Jiang, C., Zou, Y., Liu, X., Yin, Z., Dou, S., Weng, R., Cheng, W., Zhang, Q., Qin, W., Zheng, Y., Qiu, X., Huang, X. and Gui, T. (2025). *The Rise and Potential of Large Language Model Based Agents: A Survey*. Science China Information Sciences, vol. 68, no. 2, Article 121101. DOI: 10.1007/s11432-024-4222-0.
- [12] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I. and Gonzalez, J. E. (2023). *MemGPT: Towards LLMs as Operating Systems*. arXiv preprint arXiv:2310.08560.
- [13] Anthropic (2024). *Introducing the Model Context Protocol*. Anthropic News, 25 November 2024. <https://www.anthropic.com/news/model-context-protocol>.



- [14] Glimm, B., Horrocks, I., Motik, B., Stoilos, G. and Wang, Z. (2014). *HermiT: An OWL 2 Reasoner*. Journal of Automated Reasoning, vol. 53, no. 3, pp. 245–269. DOI: 10.1007/s10817-014-9305-1.
- [15] Poveda-Villalón, M., Gómez-Pérez, A. and Suárez-Figueroa, M. C. (2014). *OOPS! (Ontology Pitfall Scanner!): An On-line Tool for Ontology Evaluation*. International Journal on Semantic Web and Information Systems (IJSWIS), vol. 10, no. 2, pp. 7–34. DOI: 10.4018/ijswis.2014040102.

## A SPARQL Sorgu Kataloğu

Bu ek, Bölüm 1’de tanımlanan altı yetkinlik sorusuna karşılık gelen SPARQL sorgularını ve reasoner çıkarımlarıyla zenginleştirilmiş ontoloji üzerinde elde edilen sonuçların özetini sunmaktadır. Tüm sorgular ortak PREFIX bildirimini paylaşmaktadır:

```
1 PREFIX : <http://www.ege.edu.tr/cs/
ontology/llm-agent#>
2 PREFIX rdf: <http://www.w3.org
/1999/02/22-rdf-syntax-ns#>
```

### Q1: Araç Erişimi

```
1 SELECT ?agent ?tool ?resource WHERE {
2   ?agent rdf:type :Agent ;
3     :makesToolCall ?call .
4   ?call :invokesTool ?tool .
5   OPTIONAL { ?tool :accessesResource ?
6     resource . }
```

Sonuç: CodeExecutorAgent bireyinin iki ayrı araca (CodeInterpreterTool, WebSearchTool) ve dört farklı kaynağa (CodeResource, DBResource, FileResource, WebResource) erişimi vardır.

### Q2: Etmen Rolü

```
1 SELECT ?agent ?role WHERE {
2   ?agent rdf:type :Agent ;
3     :hasRole ?role .
4   FILTER(?role IN (:Orchestrator, :
5     Worker, :Reviewer))
6 }
```

Sonuç:

| Etmen                | Rol          |
|----------------------|--------------|
| OrchestratorAgent    | Orchestrator |
| ResearchPlannerAgent | Worker       |
| CodeExecutorAgent    | Worker       |
| ReviewerAgent        | Reviewer     |

Tablo 7: Q2 sorgusu sonucu

### Q3: Hafıza ve Etmen Tipi

```
1 SELECT ?agent ?memoryType WHERE {
2   ?agent rdf:type :Agent ;
3     :hasMemory ?mem .
4   ?mem rdf:type ?memoryType .
5   FILTER(?memoryType IN (:
6     ShortTermMemory,
7     :LongTermMemory, :EpisodicMemory,
8     :SemanticMemory))
9 }
```

Sonuç: CodeExecutorAgent bireyi epizodik, uzun vadeli ve semantik hafızaya; ResearchPlannerAgent ise kısa vadeli hafızaya sahiptir. Tam döküm ana metindeki Tablo 5’te verilmiştir.

### Q4: Görev Devri

```
1 SELECT ?delegator ?delegatee ?task
2 WHERE {
3   ?d rdf:type :Delegation ;
4     :delegator ?delegator ;
5     :delegatesTo ?delegatee ;
6     :delegatedTask ?task .
7 }
```

Sonuç:

| Devreden              | Devralan           | Görev             |
|-----------------------|--------------------|-------------------|
| Orchestrator-Agent    | Reviewer-Agent     | SummaryTask       |
| Research-PlannerAgent | CodeExecutor-Agent | CodeAnalysis-Task |

Tablo 8: Q4 sorgusu sonucu

### Q5: Oturum Mesajları

```
1 SELECT ?session ?messageId ?role ?
2   author WHERE {
3   ?session rdf:type :Session ;
4     :hasMessage ?msg .
5   ?msg :messageId ?messageId ;
6     :hasMessageRole ?role .
7   OPTIONAL { ?msg :generatedBy ?author
8     . }
```

Sonuç: Dört mesaj ve iki oturum elde edilmiştir. ResearchSession içindeki msg-001 kullanıcı mesajıdır; msg-002, ResearchPlannerAgent tarafından üretilmiştir. ExecutionSession içindeki msg-003 ve msg-004 sırasıyla CodeExecutorAgent ve ReviewerAgent tarafından üretilen Assistant rolündeki mesajlardır.

### Q6: Çok-Etmenli Koordinasyon

```
1 SELECT ?group ?member ?workflow ?
2   protocol WHERE {
3   ?group rdf:type :AgentGroup ;
4     :hasMember ?member .
5   OPTIONAL { ?group :coordinatesVia ?
6     workflow . }
7   OPTIONAL { ?workflow :
8     followsProtocol ?protocol . }
9 }
```

Sonuç: ResearchTeam grubu dört üyeden oluşmakta, ResearchWorkflow üzerinden koordine edilmekte ve MCP protokolünü izlemektedir.

## B Gerçek Framework CrewAI Eşlemesi

### Sınıflarının Ontolojiye Eşlenmesi

Ontolojinin çerçeveden bağımsızlık ve kapsama (*coverage*) iddiasını ampirik olarak sınamak amacıyla, iki yaygın BDM-etmen çerçevesi (LangChain ve CrewAI) kurulmuş ve sınıf hiyerarşileri *canlı reflection* ile çıkarılarak 37 ontoloji terimine eşlenmiştir. Eşleme, dokümantasyondan değil; kurulu paketlerden Python `importlib/inspect` aracılığıyla elde edilen gerçek `__mro__` (*method resolution order*) verisine dayanır. Üretici betik depoda `framework-mapping/reflect.py` olarak yer almaktadır.

**Ortam:** Python 3.12; langchain 1.3.2, langchain\_core 1.4.0, crewai 1.14.6.

```
1 import importlib, inspect
2 def mro_of(path):
3     mod, _, cls = path.rpartition(".")
4     obj = getattr(importlib,
5         import_module(mod), cls)
6     base = obj if inspect.isclass(obj)
7         else type(obj)
8     return [c.__qualname__ for c in
9         base.__mro__]
```

```
mro_of("langchain_core.messages.
BaseMessage")
# -> ['BaseMessage', 'Serializable', '
BaseModel', 'ABC', 'object']
```

Tablolarda modül örnekleri kısaltılmıştır: lc=langchain\_core, lg=langgraph, la=langchain.

### LangChain / LangGraph Eşlemesi

| Ontoloji Terimi                | Gerçek Sınıf                                            | Not                                      |
|--------------------------------|---------------------------------------------------------|------------------------------------------|
| Agent                          | lg.prebuilt.create_react_agent; la.agents.AgentExecutor | etmen yürütücü                           |
| AgentGroup                     | lg.graph.StateGraph                                     | çok-etmen graf                           |
| Role                           | -                                                       | birinci-sınıf yok (SystemMessage/prompt) |
| LLMModel                       | lc....BaseChatModel                                     | ← Runnable                               |
| ModelConf., Context, Parameter | lc.runnables.RunnableConfig                             | config sözlüğü                           |
| Session                        | lg.checkpoint.base.Checkpoint                           | thread durumu                            |
| Message                        | lc.messages.BaseMessage                                 | -                                        |
| MessageRole                    | AIMessage/HumanMessage/SystemMessage/ToolMessage        | oneOf'a birebir                          |
| Tool                           | lc.tools.BaseTool                                       | ← Runnable                               |
| ToolCall                       | lc.messages.tool.ToolCall                               | -                                        |
| ToolResult                     | lc.messages.ToolMessage                                 | -                                        |
| Resource                       | lc.documents.Document                                   | -                                        |
| Action                         | lc.agents.AgentAction                                   | -                                        |
| Capability, Task               | lc.runnables.Runnable                                   | LCEL: tek tip yürütülebilir              |
| ShortTermMemory                | lg.checkpoint....InMemorySaver                          | thread-kapsamlı                          |
| LongTermMemory                 | lg.store.base.BaseStore                                 | thread-ötesi                             |
| SemanticMemory                 | lc.vectorstores.VectorStore                             | gömme tabanlı                            |
| Prompt                         | lc.prompts.BasePromptTemplate                           | -                                        |
| SystemPrompt                   | lc.prompts.SystemMessage-PromptTemplate                 | -                                        |
| PromptTemplate                 | lc.prompts.ChatPromptTemplate                           | -                                        |
| Workflow                       | lg.graph.StateGraph                                     | graf akışı                               |
| Delegation                     | lg.types.Command                                        | goto ile devir                           |
| Channel                        | lg.channels.base.BaseChannel                            | dahili durum kanalı                      |

Tablo 9: LangChain/LangGraph sınıflarının ontolojiye eşlenmesi (reflection)

| Ontoloji Terimi                             | Gerçek Sınıf                                    | Not                              |
|---------------------------------------------|-------------------------------------------------|----------------------------------|
| Agent                                       | crewai.agent.core.Agent                         | ← BaseAgent                      |
| Role                                        | Agent.role (alan)                               | rol merkezi Agent alanı          |
| AgentTemplate                               | crewai.agent.core.Agent                         | sablon = Agent                   |
| AgentGroup                                  | crewai.crew.Crew                                | agents = hasMember               |
| LLMModel, ModelConf.                        | crewai.llm.LLM                                  | ← BaseLLM                        |
| Message                                     | crewai.tasks.task_output.TaskOutput             | -                                |
| Tool                                        | crewai.tools.base_tool.BaseTool                 | -                                |
| ToolCall, ToolResult                        | crewai.tools.tool_usage.ToolUsage               | araç yürütme                     |
| Resource                                    | crewai.knowledge.knowledge.Knowledge            | bilgi kaynağı                    |
| Memory                                      | crewai.memory.unified_memory.Memory             | birleşik bellek                  |
| ShortTerm/Long-Term/Episodic/SemanticMemory | Memory + MemoryScope                            | ayrı sınıf yok; hiyerarşik scope |
| MemoryStrategy                              | MemoryConfig (recency/ semantic/-consolidation) | geri çağırma/sıkıştırma          |
| Prompt, PromptTemplate                      | crewai.utilities.prompts.Prompts                | -                                |
| Task                                        | crewai.task.Task                                | async_execution ↔ ExecutionMode  |
| Subtask                                     | Task.context (alan)                             | görev bağımlılığı                |
| Plan                                        | utilities.planning_handler.-CrewPlanner         | Crew.planning                    |
| Workflow                                    | crewai.process.Process                          | {sequential, hierarchical}       |
| Workflow (olay)                             | crewai.flow.flow.Flow                           | olay-güdümlü akış                |
| Delegation                                  | crewai.tools.agent_tools.AgentTools             | Agent.allow_delegation           |
| ExecutionContext                            | crewai.crews.crew_output.CrewOutput             | -                                |

Tablo 10: CrewAI sınıflarının ontolojiye eşlenmesi (reflection)

### Bulgular

**Çerçeve bağımsızlık doğrulandı.** 37 terimin büyük çoğunluğu her iki çerçevede de somut bir sınıf veya alanla karşılanmaktadır; ancak hiçbir çerçeve terimlerin tamamını birinci-sınıf olarak içermez. Agent, Tool, Message, Memory, LLMModel, Task/Workflow ve Delegation her iki çerçevede de ortak çekirdeği oluşturur. Buna karşılık Role ve Plan yalnızca CrewAI'de, MessageRole, Channel ve ayrık hafıza katmanları ise yalnızca LangChain'de birinci-sınıftır. Her çerçevenin ortak kavram kümesinin yalnızca bir alt kümesini somutlaştırması, ontolojinin tek bir çerçevenin soyutlama tercihine bağlı olmadığını gösterir.

**Aynı kavram, farklı soyutlama.** İki çerçeve, ontolojinin var oluş gerekçesini doğrudan örnekleyen ters yönlü tasarım tercihleri sergiler. LangChain, yürütülebilir her bileşeni tek bir Runnable (LCEL) arayüzünde tekilleştirir; ontolojideki Action, Capability ve Task ayrımı burada tek tipe iner. CrewAI ise ontolojinin dört hafıza alt türünü tek bir Memory sınıfında hiyerarşik MemoryScope ve ağırlık parametreleri (recency/semantic/consolidation) altında birleştirir. Ontoloji, bu birbirine zıt soyutlamaları ortak terimlere indirgeyerek iki çerçeveyi karşılaştırılabilir kılar.

**Birebir eşleşmeler.** Bazı ontoloji terimleri çahşan sistemlerin yapı taşlarıyla bire bir örtüşmektedir: LangChain'in AIMessage/HumanMessage/SystemMessage/ToolMessage sınıfları MessageRole'un {Assistant, User, System, ToolRole} kapalı listesini; LangGraph'in checkpointer (thread-ici) ve store (thread-ötesi) ayrımı ShortTermMemory/LongTermMemory ayrımını; CrewAI'nin Task.async\_execution alanı ise ExecutionMode{Sync, Async} kapalı listesini doğrudan karşılar. Bu örtüşmeler, terimlerin yapay

değil, gerçek mimari kararlara dayandığını gösterir.

**Kapsanmayanlar.** `Protocol` her iki çerçevede de birinci-sınıf değildir; `Channel` yalnızca `LangGraph`'te dahili durum kanalı olarak bulunur. `Observation`, `Policy` ve `EpisodicMemory` gibi terimler ise olay (*event*) ve guardrail mekanizmalarıyla yalnızca dolaylı karşılanır. Bu terimlerin ontolojide birinci-sınıf tutulması, modelin mevcut çerçevelerin ötesindeki tasarım uzayını da kapsadığını ve bir üst-model (*meta-model*) olma iddiasını desteklediğini gösterir.



## C Reasoner Çıkarımları

HermiT reasoner, `ClassAssertion` ve `InverseObjectProperties` üreteçleriyle çalıştırılmış; sonuçta materializasyon edilen ontoloji üzerinde aşağıdaki yeni `rdf:type` aksiyomları tespit edilmiştir:

| Birey                          | Çıkarılan Tanımlı Sınıf           |
|--------------------------------|-----------------------------------|
| <code>CodeExecutorAgent</code> | <code>AutonomousAgent</code>      |
| <code>CodeExecutorAgent</code> | <code>MemoryAugmentedAgent</code> |

Tablo 11: Reasoner tarafından üretilen sınıf üyelikleri

`ResearchPlannerAgent` bireyinin `PlanningAgent` sınıfına eklenmemesi, tanımdaki  $\neg(\exists \text{makesToolCall} . \text{ToolCall})$  negasyonunun Açık Dünya Varsayımı altında kapatılamamasından kaynaklanmaktadır. Birey düzeyinde kapama için ABox'a *ResearchPlannerAgent* :  $\neg(\exists \text{makesToolCall} . \text{ToolCall})$  biçiminde bir class-assertion (OWL: `ClassAssertion(ObjectComplementOf(ObjectSomeValuesFrom(:makesToolCall :ToolCall)) :ResearchPlannerAgent)`) eklendiğinde reasoner negasyonu çıkarsayarak ilgili sınıflandırmayı üretir. Bu kapama, Reasoner Tutarlılık Kontrolü bölümünde deneysel olarak gösterilmiştir: aksiyom eklenip ontoloji HermiT ile yeniden çıkarıma sokulduğunda `ResearchPlannerAgent`, `PlanningAgent` olarak sınıflandırılmıştır. Söz konusu yerel kapama, ontolojinin sonraki sürümlerinde değerlendirilecek bir esnetme noktasıdır.

Reasoner ayrıca tanımlanan `owl:inverseOf` çiftleri üzerinden `capabilityOf`, `memberOf`, `partOfSession`, `observedIn`, `playedBy` ve `generatedBy` özelliklerine ilişkin ters yön aksiyomlarını otomatik olarak çıkarsamıştır. Bu çıkarımlar, ABox'taki üyelik ve oturum ilişkilerinin tek yönlü bildirilmiş olmasına karşın iki yönlü olarak sorgulanabilmesini sağlamaktadır.